


**DATA STRUCTURES AND ALGORITHMS**

**23CSE203**

**LAB MANUAL**

| | |
|---|---|
| **Submmited by** | |
| **Name** | **A.YESWANTH** |
| **Roll No** | **AV>SC>U4CSE25003** |
| **Year/sem/section** | **2nd year/sem-3/CSE-A** |
| **Credits** | **5** |

| | |
|---|---|
| **Submmited to** | |
| **Name** | **DR.RAJKUMAR BATCHU** |
| **Department** | **School of computing** |
| **Designation** | **Asst.professor(SI.Grade)** |

# Index


| S.NO | TITLE | DATE | PG. NO. |
|---|---|---|---|
| 1. | 1. Recursive countdown from n to Launch.<br>2. Recursive calculation of compound growth factor.<br>3. Recursive search for employee ID in list.<br>4. Recursive factorial calculation for parcel arrangements.<br>5. Recursive Fibonacci series generation. | 23-07-26 | 5-14 |
| 2. | **1)** write a python program to perform linear search on the given input.<br>**2.a)**write a python program to perform binary search on the given input verify by taking sorted array.<br>**2.b)** write a python program to perform binary search on the given input verify by taking unsorted array. | 30-07-26 | 15-21 |
| 3. | 1.a) Write a python program to sort the elements using bubble sort technique. Take an input from the user.<br>1.b) Write a python program to sort the elements using insertion sort technique. Take an input from the user.<br>1.c) Write a python program to sort the elements using selection sort technique. Take an input from the user.<br>1.d) Write a python program to sort the elements using quick sort technique. Take an input from the user.<br>1.e) Write a python program to sort the elements using merge sort technique. Take an input from the user. | 06-08-26 | 22-35 |
| 4. | Aim 4.a:Write a program to implement a single linked list which would perform the below operations, by taking an input from the user.<br>a)create a linked list<br>b)insert at beginning<br>c)insert at end<br>d)insert at a specified index<br>e)delete by value<br>f)delete 1st node | 13-08-26 | 36-68 |

| | | | |
|---|---|---|---|
| | g)delete last node<br>h)count no.of nodes<br>i)display or traverse<br>j)search for a number<br>k)exit<br><br>Aim 4.b:Write a program to implement a double linked list which would perform the below operations, by taking an input from the user.<br>a)create a linked list<br>b)insert at beginning<br>c)insert at end<br>d)insert at a specified index<br>e)delete by value<br>f)delete 1st node<br>g)delete last node<br>h)count no.of nodes<br>i)display or traverse<br>j)search for a number<br>k)exit<br>Aim 4.c:Write a program to implement a circular linked list which would perform the below operations, by taking an input from the user.<br>a)create a linked list<br>b)insert at beginning<br>c)insert at end<br>d)insert at a specified index<br>e)delete by value<br>f)delete 1st node<br>g)delete last node<br>h)count no.of nodes<br>i)display or traverse<br>j)search for a number<br>k)exit | | |

| | | | |
|---|---|---|---|
| | | | |
| 5. | **Week 5:** Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements. | 27-08-26 | 69-79 |
| 6. | **Week 6:** Write a python program to implement a queue using an Array, Linked list, circular queue. Implement the queue operations such as enqueue, dequeue, peek and also display the queue elements. | 03-09-26 | 80-93 |
| 7. | **Week 7:** a)write a python program to implement binary tree search using an array with all its operations.<br>b)write a python program to implement binary tree using an linked list with all its operations. | 24-09-26 | 94-106 |
| 8. | **Week 8:** write a python program to implement Binary tree with its operations using array and linked list. | 08-10-26 | 107-113 |
| 9. | **Week 9:** write a python code for AVL tree using array and linked list with all its operations. | 22-10-26 | 114-141 |
| 10. | Hashing – Separate Chaining, Linear Probing and Quadratic Probing | 29-10-26 | 142-148 |

1A).AIM:A Rocket launch system displays a countdown before launch. Instead of suing a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by “launch”.

Code:

```
def countdown(n):
    if n < 0:
        print("Invalid input")
        return
    if n == 0:
        print("launch")
        return
    print(n)
    countdown(n - 1)

n = int(input("Enter n: "))
countdown(n)
```

output:




**Explanation**

**Condition 1: Positive Input (n > 0)**

- The function checks whether n == 0.
- Since n is positive, the condition is **False**.
- It executes:
  - print(n) → prints the current value of n.
  - countdown(n-1) → calls the same function again with n-1.
- This process continues until n becomes 0.
- When n == 0, the condition becomes **True**.

- It prints **launch**.
- The return statement stops that function call.
- Then the recursive calls return one by one.
- Since there is no value returned by the function, it returns **None**.

**Condition 2: Zero Input (n == 0)**

- The function checks if n == 0.
- The condition is immediately **True**.
- It prints **launch**.
- The return statement stops the function.
- No recursive call takes place.
- The function returns **None**.

**Condition 3: Negative Input (n < 0)**

- The condition n == 0 is **False**.
- It prints the negative value.
- Then it calls itself with n-1, making the value even more negative.
- Therefore, it will **never reach 0**.
- The recursion continues indefinitely.
- Eventually, Python raises a **RecursionError** because the maximum recursion depth is exceeded

| S.no | ERROR OBTAINED | RECTIFICTION |
|---|---|---|
| **1** | Negative input in countdown | Add a condition to handle negative numbers before recursion |

1B). Aim: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p(n) where p=principal growth, n= no. of years.

Code: def compound_interest(p, n):

```
    if n == 0:
        return 1
    else:
        return p * compound_interest(p, n - 1)

p = int(input("Enter principal value: "))
n = int(input("Enter number of years: "))

print("Result:", compound_interest(p, n))
```

output:

```
================================================== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week1b.py
Enter principal value: 10
Enter number of years: 7
Result: 10000000
```

**Explanation**

**Condition 1: n > 0**

- The function compound_interest(p, n) is called with the principal amount p and number of years n.
- The function checks whether n == 0.
- Since n is greater than 0, the condition is **False**.
- It executes the else block.
- It calculates:
  - p * compound_interest(p, n-1)
- The function calls itself recursively with n-1.
- This process continues until n becomes 0.
- Each recursive call multiplies p one more time.
- Therefore, the function calculates $\mathbf{p^n}$.

**Condition 2: n == 0**

- When n becomes 0, the condition n == 0 becomes **True**.
- The function returns 1.
- This 1 acts as the **base case** for the multiplication.
- The recursive calls then return one by one.

- Each returned value is multiplied by p.
- Finally, the result is printed using print(compound_interest(p,n))

| S.no | ERROR OBTAINED | RECTIFICTION |
|---|---|---|
| 1 | Wrong indentation | Maintain proper indentation |

1C). AIM . An Hr department stores employee IDs in a list .Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list.

Code :
```
def search_employee(emp_list, emp_id, index):
    if index == len(emp_list):
        return False

    if emp_list[index] == emp_id:
        return True

    return search_employee(emp_list, emp_id, index + 1)


emp_list = [101, 102, 103, 104, 105]

emp_id = int(input("Enter employee ID to search: "))

if search_employee(emp_list, emp_id, 0):
    print("Employee ID found")
else:
```

print("Employee ID not found")

**output:**

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week1c.py ==========
Enter employee ID to search: 103
Employee ID found
```

**Explanation**

**Condition 1: index == len(emp_list)**

- The function checks whether index has reached the length of the employee list.
- If index == len(emp_list), it means we have checked **all employee IDs**.
- The employee ID was not found.
- The function returns **False**.
- No further recursion takes place.

**Condition 2: emp_list[index] == emp_id**

- The function checks whether the current employee ID is equal to the ID we are searching for.
- If both IDs are equal, the condition becomes **True**.
- The function returns **True**.
- The recursion stops because the employee ID has been found.

**Condition 3: Employee ID does not match**

- If the current employee ID is not equal to emp_id, the condition is **False**.
- The function calls itself recursively with:
  - index + 1
- This moves to the **next employee ID** in the list.
- The process continues until:
  - The employee ID is found, or
  - The entire list is checked.

Error table:

| | | |
|---|---|---|

| S.no | ERROR OBTAINED | RECTIFICTION |
|---|---|---|
| 1 | **Missing return statement** | **Add the required return statement** |

1D. AIM. In a logistics company , the no . of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code : def factorial(n):

```
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
```

```
n = int(input("Enter a number: "))

if n < 0:
    print("Factorial is not defined for negative numbers.")
else:
    print("Factorial:", factorial(n))
```

output:

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week1d.py ==========
Enter a number: 100
Factorial: 933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272237582511852109168640000000000000000000000
```

**Explanation**

**Condition 1: Negative Input (n < 0)**

- The program first checks whether n < 0.

- If n is negative, the condition is **True**.
- Factorial is not defined for negative numbers.
- It prints:
  Factorial is not defined for negative numbers.
- The factorial() function is **not called**.

**Condition 2: Base Case (n == 0 or n == 1)**

- Inside the factorial() function, it checks:
  n == 0 or n == 1
- If n is 0 or 1, the condition is **True**.
- The function returns **1**.
- This is the **base case** of the recursion.
- It stops further recursive calls.

**Condition 3: n > 1**

- If n is greater than 1, the condition n == 0 or n == 1 is **False**.
- The else block is executed.
- It returns:
  n * factorial(n - 1)
- The function calls itself with n - 1.
- This continues until n becomes 1.
- Then the returned values are multiplied together

Error table:

| S.no | ERROR OBTAINED | RECTIFICTION |
|---|---|---|
| 1 | **Wrong recursive value** | **Updated the recursive value correctly** |

1E). AIM: Write a recursive program to print the first n terms of the fibonacci series

Code : def fibonacci(n):

```
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)


n = int(input("Enter number of terms: "))

for i in range(n):
    print(fibonacci(i), end=" ")
```

output:

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week1e.py ==========
Enter number of terms: 10
0 1 1 2 3 5 8 13 21 34
```

**Explanation**

**Condition 1: n == 0**

- The function checks whether n == 0.
- If the condition is **True**, the function returns 0.
- This is the **first base case**.
- No further recursive call takes place.

**Condition 2: n == 1**

- If n is not 0, the function checks whether n == 1.
- If the condition is **True**, the function returns 1.
- This is the **second base case**.
- No further recursive call takes place

**Condition 3: n > 1**

- If n is neither 0 nor 1, both conditions are **False**.
- The else block is executed.
- The function calls itself **two times**:
  - fibonacci(n - 1)
  - fibonacci(n - 2)
- The two returned values are added together.
- This process continues until the base cases n == 0 and n == 1 are reached.

Error table:

| S.no | ERROR OBTAINED | RECTIFICTION |
|---|---|---|
| 1 | **Incorrect Fibonacci recursion** | **Use fibonacci(n-1) + fibonacci(n-2).** |

WEEK-2

2a. AIM: Write a python program to perform linear search on the give input.

Code :
```
def linear_search(arr, k):
    for i in range(len(arr)):
        if arr[i] == k:
            return i
    return -1


arr = list(map(int, input("Enter elements: ").split()))
k = int(input("Enter element to search: "))
```

```
result = linear_search(arr, k)


if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

Output:

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week2a.py
Enter elements: 10 20 30 40
Enter element to search: 40
Element found at index: 3
```

**Explanation** :

**Condition 1: Element is Found (arr[i] == k)**

- The function linear_search(arr, k) receives the array arr and the search element k.
- The for loop starts from the first element of the array.
- It checks each element one by one using:
  arr[i] == k
- If the current element is equal to k, the condition becomes **True**.
- The function immediately returns the **index i**.
- Since the function returns the index, the loop stops immediately.
- The remaining elements are not checked.

**Condition 2: Element is Not Found**

- The loop checks every element in the array.
- If none of the elements is equal to k, the if condition is always **False**.
- After the loop finishes, the statement return -1 is executed.
- -1 indicates that the search element is **not present** in the array.

Error table:

| S.no | ERROR OBTAINED | RECTIFICTION |
|---|---|---|
| 1 | **Wrong loop range** | **Use range(n) to process exactly n elements.** |

2b. AIM: Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

Code :
```
def binary_search(arr, k):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == k:
            return mid
        elif k < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1


arr = list(map(int, input("Enter sorted elements: ").split()))
k = int(input("Enter element to search: "))
```

```
result = binary_search(arr, k)


if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

Output:

```
================================================== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week2b.py =
Enter sorted elements: 10 20 30 40 70
Enter element to search: 70
Element found at index: 4
```

**Explanation**

**Condition 1: k == arr[mid] — Element Found**

- The function binary_search(arr, k) receives the array and the search element.
- low is initialized to 0, which represents the first index.
- high is initialized to len(arr)-1, which represents the last index.
- The while loop runs as long as low <= high.
- mid is calculated using:
  mid = (low + high) // 2
- The middle element is compared with the search element k.
- If k == arr[mid], the element is found.
- The function returns the **index mid** and stops searching.

**Condition 2: k < arr[mid] — Search Left Half**

- If k is smaller than the middle element, the condition is **True**.
- This means the search element can only be present in the **left half**.
- Therefore:
  high = mid - 1
- The right half is eliminated.
- The process continues with the remaining left half

**Condition 3: k > arr[mid] — Search Right Half**

- If k is greater than the middle element, the elif condition is **False**.
- The else block is executed.
- This means the search element can only be present in the **right half**.
- Therefore:
  low = mid + 1
- The left half is eliminated.
- The process continues with the remaining right half.

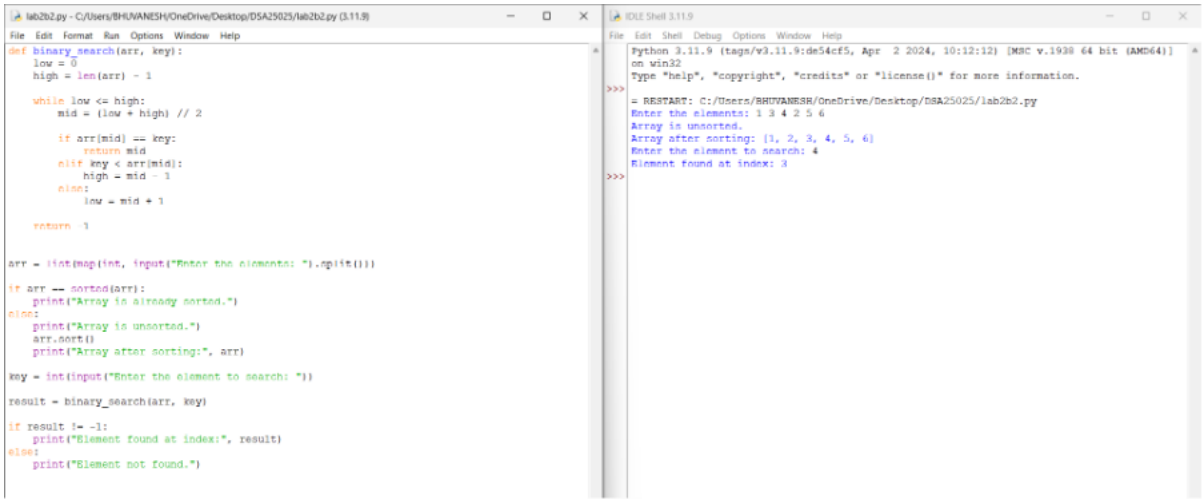
**Condition 4: low > high — Element Not Found**

- The while loop continues until low <= high becomes **False**.
- When low > high, there are no elements left to search.
- This means the search element is not present in the array.
- The function returns **-1**.

Error table:

| S.no | ERROR OBTAINED | RECTIFICTION |
|---|---|---|
| 1 | Wrong index in binary search | Use high = mid - 1 for the left half and low = mid + 1 for the right half. |

Output:




**Explanation**

**Condition 1: arr[mid] == key — Element Found**

- The function first calculates the middle index using mid = (low + high) // 2.
- It compares the middle element arr[mid] with the search element key.
- If arr[mid] == key, the condition is **True**.
- The element is found.
- The function returns the index mid.
- The search stops.

**Condition 2: key < arr[mid] — Search Left Half**

- If key is smaller than arr[mid], the condition is **True**.
- This means the required element may be present in the **left half**.
- The right half is ignored.
- high is changed to mid - 1.
- The search continues in the left half.

**Condition 3: key > arr[mid] — Search Right Half**

- If key is greater than arr[mid], the previous conditions are **False**.
- The element may be present in the **right half**.
- The left half is ignored.
- low is changed to mid + 1.
- The search continues in the right half.

**Condition 4: low > high — Element Not Found**

- The while low <= high loop continues until there are elements left to search.
- When low > high, the loop stops.
- This means the element is not present in the array.
- The function returns -1.
- The program prints **"Element not found."**

**Condition 5: Array is Already Sorted**

- The program checks whether the given array is already sorted.
- It uses:
  arr == sorted(arr)
- If the condition is **True**, it prints:
  **"Array is already sorted."**
- The array is directly used for binary search.

**Condition 6: Array is Unsorted**

- If arr == sorted(arr) is **False**, the array is unsorted.
- The program prints:
  **"Array is unsorted."**
- It then uses arr.sort() to sort the array.
- The sorted array is displayed.
- Binary search is then performed on the sorted array.

Error table:

| S.no | ERROR OBTAINED | RECTIFICTION |
|---|---|---|
| 1 | **Binary search on unsorted array** | **sort the array first using arr.sort() or provide a sorted array.** |

WEEK - 3

3a. AIM: Write a program to perform Bubble Sort technique on the user given input.

PROGRAM:

```
n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    x = int(input("Enter element: "))
    arr.append(x)

for i in range(n - 1):
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted array:", arr)
```

OUTPUT:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.

= RESTART: C:\Users\Yeswanth\AppData\Local\Programs\Python\Python314\week3a.py =
Enter number of elements: 5
Enter element: 6
Enter element: 7
Enter element: 4
Enter element: 8
Enter element: 2
Sorted array: [2, 4, 6, 7, 8]
```

EXPLANATION:

1. Start by taking input size,

The program first asks how many elements (n) the user wants to sort. This sets the boundary for loops.

2. Input the elements into the list,

The user enters n numbers, which are stored in the list arr.

3. Begin Bubble Sort process,

Sorting starts with two nested loops:

o Outer loop (i) ? Controls the number of passes. Runs n-1 times because after each pass one element is placed correctly at the end.

o Inner loop (j) ? Compares adjacent elements in each pass.

4. Condition check inside inner loop,

" The condition is: if the current element is greater than the next element (arr[j] > arr[j+1]).

" If this condition is true, the two elements are swapped.

" If the condition is false, no swap is made and the loop continues.

5. Effect of condition and swapping,

o Each time the condition is true, larger elements move step by step toward the end of the list.

o After the first pass, the largest element is at the last position.

o After the second pass, the second-largest is in its correct position, and so on.

6. Completion of sorting,

When all passes are finished, the list is sorted in ascending order. The program then prints the sorted list.

Error table:

| sno | Error obtained | Rectification |
|---|---|---|
| 1 | IndexError: list index out of range | Use correct loop range |

3b.Aim. write a python program to sort the elements using a insertion sort technique. Take an input elements from the user.

Code: n = int(input("Enter number of elements: "))

```
arr = []

for i in range(n):
    x = int(input("Enter element: "))
    arr.append(x)

for i in range(1, n):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j = j - 1

    arr[j + 1] = key

print("Sorted array:", arr)
```

Output:

```
========== RESTART: C:\Users\Yeswanth\AppData\Local\Programs\Python\Python314\week3b.py ==
Enter number of elements: 5
Enter element: 7
Enter element: 9
Enter element: 7
Enter element: 5
Enter element: 4
Sorted array: [4, 5, 7, 7, 9]
```

Explanation:

**Condition 1: for i in range(1, len(arr)) — Selecting Element**

- The loop starts from the **second element**.
- The first element is considered already sorted.
- i represents the element that needs to be placed correctly.

**Condition 2: key = arr[i] — Storing Element**

- The current element is stored in key.

```
key = arr[i]
```

- This element will be compared with the elements before it.

**Condition 3: j = i - 1 — Getting Previous Position**

- j points to the element just before the current element.

```
j = i - 1
```

**Condition 4: while j >= 0 and key < arr[j] — Comparing Elements**

- The current element is compared with the previous elements.
- If the previous element is greater than key, it is moved one position to the right.

```
while j >= 0 and key < arr[j]:
   arr[j + 1] = arr[j]
   j -= 1
```

- This creates space for the current element.

**Condition 5: arr[j + 1] = key — Inserting Element**

- The key is placed in its correct position.

```
arr[j + 1] = key
```

- The elements before the current position are now sorted.

**Condition 6: return arr — Returning Sorted Array**

- After all elements are processed, the function returns the sorted array.

```
return arr
```

error table:

| sno | Error obtained | Rectification |
|---|---|---|
| 1 | Elements not sorted correctly | Shift elements before insertion |

3c.write a python program to sort the elements using selection sort technique . take an input from the user.

Code:

```
n = int(input("Enter number of elements: "))


arr = []


for i in range(n):
    x = int(input("Enter element: "))
    arr.append(x)


for i in range(n - 1):
    min_index = i


    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j


    arr[i], arr[min_index] = arr[min_index], arr[i]


print("Sorted array:", arr)
```

Output:

```
========== RESTART: C:\Users\Yeswanth\AppData\Local\Programs\Python\Python314\week3b.py ==
Enter number of elements: 5
Enter element: 7
Enter element: 9
Enter element: 7
Enter element: 5
Enter element: 4
Sorted array: [4, 5, 7, 7, 9]
```

Explanation:

**Condition 1: for i in range(n) — Selecting Position**

- The loop goes through each position in the array.
- i represents the current position where the smallest element will be placed.

**Condition 2: mini = i — Assuming Minimum**

- Initially, the element at index i is assumed to be the smallest.

**Condition 3: for j in range(i+1,n) — Finding Minimum**

- The loop checks all elements after position i.
- It searches for the smallest element.

**Condition 4: if arr[mini] > arr[j] — Comparing**

- Compares the current minimum with arr[j].
- If arr[j] is smaller, mini is updated.

**Condition 5: mini = j — Updating Minimum**

- Stores the index of the newly found smallest element.

**Condition 6: arr[mini], arr[i] = arr[i], arr[mini] — Swapping**

- The smallest element is swapped with the element at position i.
- This places the smallest element in its correct position.

**Condition 7: return arr — Returning Result**

- Returns the completely sorted array.

Error table:

| Sno | Error obtained | Rectification |
|---|---|---|
| 1 | Wrong minimum value | Update min_index properly |

3d.write a python program to sort the elements using the merge sort technique . take an input from the user.

Code:

```
left = merge_sort(arr[:mid])
```

```
    right = merge_sort(arr[mid:])

    merged = []

    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i = i + 1
        else:
            merged.append(right[j])
            j = j + 1

    while i < len(left):
        merged.append(left[i])
        i = i + 1

    while j < len(right):
        merged.append(right[j])
        j = j + 1

    return merged
n = int(input("Enter number of elements: "))
arr = []
for i in range(n):
    x = int(input("Enter element: "))
    arr.append(x)
```

```
arr = merge_sort(arr)


print("Sorted array:", arr)
```

Output:

```
========== RESTART: C:\Users\Yeswanth\AppData\Local\Programs\Python\Python314\week3b.py ==
Enter number of elements: 5
Enter element: 7
Enter element: 9
Enter element: 7
Enter element: 5
Enter element: 4
Sorted array: [4, 5, 7, 7, 9]
```

Explanation:

**Condition 1: if len(arr) <= 1 — Checking the Base Condition**

- The function first checks whether the array contains **one or zero elements**.

```
if len(arr) <= 1:
    return arr
```

- If the array has one or zero elements, it is already sorted.
- No further division or comparison is required.
- The function returns the array as it is:

```
return arr
```

**Condition 2: mid = len(arr) // 2 — Finding the Middle**

- The array is divided into two parts.
- The middle position is calculated using:

```
mid = len(arr) // 2
```

- len(arr) gives the total number of elements.
- // 2 performs integer division and finds the middle index.

**Condition 3: left = arr[:mid] — Creating the Left Part**

- The elements before the middle position are stored in left.

```
left = arr[:mid]
```

**Condition 4: right = arr[mid:] — Creating the Right Part**

- The elements from the middle position to the end are stored in right.

right = arr[mid:]

**Condition 5: merge_sort(left) and merge_sort(right) — Sorting Both Parts**

- The left and right parts are recursively sorted.

merge_sort(left)

merge_sort(right)

- The array continues to be divided until each part contains only **one element**.
- A single element is already sorted.
- The smaller sorted parts are then merged together.

**Condition 6: i = j = k = 0 — Initializing Indexes**

- Three variables are initialized:

i = j = k = 0

- i keeps track of the current element in the **left array**.
- j keeps track of the current element in the **right array**.
- k keeps track of the position in the **original array** where the element will be placed.

**Condition 7: while i < len(left) and j < len(right) — Comparing Both Parts**

- The program compares elements from the left and right arrays.

while i < len(left) and j < len(right):

- The loop continues as long as both arrays still have elements to compare.
- This allows the program to select the smaller element and place it into the original array.

**Condition 8: if left[i] < right[j] — Comparing Elements**

- The current elements of the left and right arrays are compared.

if left[i] < right[j]:

- If the left element is smaller than the right element, it is placed into the original array.

arr[k] = left[i]

- Then i is increased:

i += 1

- This moves to the next element of the left array.

**Condition 9: else — Taking the Right Element**

- If the right element is smaller than or equal to the left element, the right element is placed into the original array.

```
else:
    arr[k] = right[j]
```

- Then j is increased:

```
j += 1
```

- This moves to the next element of the right array.

**Condition 10: k += 1 — Moving to the Next Position**

- After placing an element into the original array, k is increased.

```
k += 1
```

**Condition 11: while i < len(left) — Copying Remaining Left Elements**

- Sometimes the right array becomes empty before the left array.

```
while i < len(left):
```

- Any remaining elements in the left array are copied into the original array.

```
arr[k] = left[i]
```

- Then both i and k are increased:

```
i += 1
k += 1
```

- The remaining elements are already sorted, so no comparison is needed.

**Condition 12: while j < len(right) — Copying Remaining Right Elements**

- Sometimes the left array becomes empty before the right array.

```
while j < len(right):
```

- Any remaining elements in the right array are copied into the original array.

```
arr[k] = right[j]
```

- Then j and k are increased:

```
j += 1
k += 1
```

- This completes the merging process.

Error table:

| sno | Error obtained | Rectification |
|---|---|---|
| 1 | Recursive error | Use if len(arr) <= 1 |
| 2 | IndexError | Check left/right array indices |

Bottom of Form

3e. Aim. Write a python program to sort the elements using a quick sort technique . take an input elements from the user

Code:

```
def quicksort(a, low, high):
    if low < high:

        pivot = a[low]

        i = low
        j = high
while i < j:
while i <= high and a[i] <= pivot:
            i = i + 1
while j >= low and a[j] > pivot:
            j = j - 1

          if i < j:
            a[i], a[j] = a[j], a[i]

        a[low], a[j] = a[j], a[low]

        quicksort(a, low, j - 1)
```

```
    quicksort(a, j + 1, high)
a = list(map(int, input("Enter elements separated by spaces: ").split()))
n = len(a)
quicksort(a, 0, n - 1)
print("Sorted array:", a)
```

Output:

```
================================================== RESTART: C:\Users\Yeswanth\AppData\Local\Programs\Python\Python314\week3b.py ==
Enter number of elements: 5
Enter element: 7
Enter element: 9
Enter element: 7
Enter element: 5
Enter element: 4
Sorted array: [4, 5, 7, 7, 9]
```

**Explanation**

**Condition 1: if arr[j] < pivot — Element Smaller Than Pivot**

- The function partition(arr, low, high) is used to divide the array into two parts.
- pivot = arr[high] selects the **last element** of the array as the pivot.
- i = low - 1 is initialized to keep track of the position where elements smaller than the pivot should be placed.
- The for loop checks each element from index low to high-1.
- The condition arr[j] < pivot checks whether the current element is smaller than the pivot.
- If the condition is **True**, i is increased by 1.
- The current element is then swapped with the element at index i:

```
arr[i], arr[j] = arr[j], arr[i]
```

- This places all elements smaller than the pivot toward the left side.

**Condition 2: arr[j] >= pivot — No Swap Required**

- If arr[j] < pivot is **False**, it means the current element is greater than or equal to the pivot.
- No swapping is performed.
- The loop continues to check the next element.
- These elements will remain on the right side of the pivot after partitioning.

**Condition 3: arr[i+1], arr[high] = arr[high], arr[i+1] — Placing the Pivot**

- After all elements have been checked, the pivot is placed in its correct position.

- The pivot is swapped with the element at index i+1:

arr[i+1], arr[high] = arr[high], arr[i+1]

- After this operation:
  - Elements smaller than the pivot are on the **left**.
  - The pivot is in its **correct position**.
  - Elements greater than or equal to the pivot are on the **right**.
- The function returns the pivot position:

return i+1

**Condition 4: if low < high — Continue Quick Sort**

- The function quick_sort(arr, low, high) is used to sort the array.
- The condition low < high checks whether the current part of the array contains more than one element.
- If the condition is **True**, the array needs to be sorted.
- The partition() function is called:

pi = partition(arr, low, high)

- pi stores the final position of the pivot.

**Condition 5: quick_sort(arr, low, pi-1) — Sort Left Part**

- After partitioning, the elements on the left side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, low, pi-1)

- The pivot itself is not included because it is already in its correct position.

**Condition 6: quick_sort(arr, pi+1, high) — Sort Right Part**

- The elements on the right side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, pi+1, high)

- Again, the pivot is not included because it is already correctly positioned.

**Input Section**

- n = int(input("enter the no of elements:")) takes the number of elements from the user.
- arr = [] creates an empty list to store the elements.

- The for loop runs n times and takes each element from the user:
- for i in range(n):

arr.append(int(input("enter the elements:")))

- Each entered value is converted into an integer and added to the array using append().

**Calling the Quick Sort Function**

quick_sort(arr, 0, n-1)

- This calls the quick_sort() function.
- 0 represents the first index of the array.
- n-1 represents the last index of the array.
- The complete array is therefore passed for sorting.

.

Error table:

| sno | Error obtained | Rectification |
|---|---|---|
| 1 | IndexError | Check i, j, and pivot boundaries |
| 2 | Wrong swapping conditions | By shifting to the correct spaces |

# WEEK-4

AIM: write a python program to implement a single linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or travsere

x. exit

code: class Node:

```
    def __init__(self, data):
        self.data = data
        self.next = None



class SLL:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next
```

```
            temp.next = new_node

    def insert_beginning(self):
        data = int(input("Enter data: "))

        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node

    def insert_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))
```

```
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        return

    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Index out of range")
            return
        temp = temp.next

    if temp is None:
        print("Index out of range")
        return

    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node

def delete_value(self):
    value = int(input("Enter value to delete: "))
```

```
    if self.head is None:
        print("List is empty")
        return

    if self.head.data == value:
        self.head = self.head.next
        return

    temp = self.head

    while temp.next is not None:
        if temp.next.data == value:
            temp.next = temp.next.next
            return
        temp = temp.next

    print("Value not found")

def delete_first(self):
    if self.head is None:
        print("List is empty")
    else:
        self.head = self.head.next

def delete_last(self):
    if self.head is None:
        print("List is empty")
        return
```

```
        if self.head.next is None:
            self.head = None
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None

    def count(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head
```

```
        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


sll = SLL()

while True:
    print("\n--- SINGLY LINKED LIST ---")
    print("1. Create")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specific Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()

    elif choice == 2:
        sll.insert_beginning()
```

```
elif choice == 3:
    sll.insert_end()

elif choice == 4:
    sll.insert_index()

elif choice == 5:
    sll.delete_value()

elif choice == 6:
    sll.delete_first()

elif choice == 7:
    sll.delete_last()

elif choice == 8:
    sll.count()

elif choice == 9:
    sll.display()

elif choice == 10:
    print("Program ended")
    break

else:
    print("Invalid choice")
```

Output:

Creating a linked list

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter value: 1
Enter value: 2
Enter value: 3
```

Inserting at the begininnig

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter value: 6
```

Insertion at the end:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter value: 5
```

Insertion at specific index:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter value: 2
Enter index: 2
```

Delete at beginning:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
```

Delete at the end:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
```

Delete at the specific index:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 5
```

Number of nodes:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 3
```

Display all the nodes:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
1 -> 2 -> 2 -> None
```

Exiting the program:

```
----- SINGLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.
```

Explanation:

1. A **singly linked list** is a collection of nodes where each node contains **two parts**: data and a link to the next node.
2. Each node contains:
   - **Data** – stores the value.
   - **Next (next)** – stores the reference/address of the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.

6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are created one by one and added to the list.
7. For the first node, both **head and tail** point to the same node, and the next of the tail is None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node's next points to the current head, and the new node becomes the new head.
9. During **insertion at the end**, the new node is added after the current tail. The current tail's next points to the new node, and the new node becomes the new tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the appropriate nodes by adjusting their next links.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by making the previous node point to the node after the deleted node.
12. If the node to be deleted is the **head**, the next node becomes the new head.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
16. During **deletion at the end**, the list is traversed to find the node before the tail. That node becomes the new tail and its next is set to None.
17. The **count operation** traverses the list from the head to the end and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves through the list using the next link until None is reached.
20. The program continues to execute until the user selects the **Exit** option.

Error table:

| sno | Error obtained | rectification |
|---|---|---|
| 1 | Infinite loop or incomplete traversal due to not updating current | Move to the next node using temp = temp.next |

| | | |
|---|---|---|
| 2 | Forgetting to update head while inserting/deleting at the beginning | Update head to point to the new first node using head = new_node or head = head.next |
| | | |

Week 4 b

AIM: write a python program to implement a double linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or travsere

x. exit

code:

*dll.py - C:/Users/SREEHAAS/OneDrive/Desktop/dsa lab/week4/dll.py (3.13.7)*

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for _ in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)

        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node

        self.head = new_node

    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            return

        current = self.head

        while current.next:
            current = current.next

        current.next = new_node
        new_node.prev = current
```

```python
# iv. Insert at specific index
def insert_at_index(self, value, index):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_beginning(value)
        return

    current = self.head

    for _ in range(index - 1):
        if current is None:
            print("Index out of range")
            return
        current = current.next

    if current is None:
        print("Index out of range")
        return

    new_node = Node(value)

    new_node.next = current.next
    new_node.prev = current

    if current.next is not None:
        current.next.prev = new_node

    current.next = new_node
```

```
# v. Delete by value
def delete_by_value(self, value):
    if self.head is None:
        print("List is empty")
        return

    current = self.head

    while current and current.data != value:
        current = current.next

    if current is None:
        print("Value not found")
        return

    if current.prev is not None:
        current.prev.next = current.next
    else:
        self.head = current.next

    if current.next is not None:
        current.next.prev = current.prev

# vi. Delete at beginning
def delete_beginning(self):
    if self.head is None:
        print("List is empty")
        return

    self.head = self.head.next

    if self.head is not None:
        self.head.prev = None
```

```
    # vii. Delete at end
    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while current.next:
            current = current.next

        if current.prev is None:
            self.head = None
        else:
            current.prev.next = None

    # viii. Count nodes
    def count_nodes(self):
        count = 0
        current = self.head

        while current:
            count += 1
            current = current.next

        return count

    # ix. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while current:
            print(current.data, end=" <-> ")
            current = current.next

        print("None")


dll = DoublyLinkedList()
```

```
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        value = int(input("Enter value: "))
        dll.insert_beginning(value)

    elif choice == 3:
        value = int(input("Enter value: "))
        dll.insert_end(value)

    elif choice == 4:
        value = int(input("Enter value: "))
        index = int(input("Enter index: "))
        dll.insert_at_index(value, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        dll.delete_by_value(value)

    elif choice == 6:
        dll.delete_beginning()

    elif choice == 7:
        dll.delete_end()

    elif choice == 8:
        print("Number of nodes:", dll.count_nodes())

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

Output:

Creating a linked list:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter value: 1
Enter value: 2
Enter value: 3
```

Inserting at the beginning:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter value: 6
```

Insert at the end:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter value: 8
```

Insert at the specific index:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter value: 44
Enter index: 4
```

Delete By value

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 44
```

Delete at the beginning:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
```

Delete at the end:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
```

Number of nodes:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 3
```

Display the nodes:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
1 <-> 2 <-> 3 <-> None
```

Exiting the program:

```
----- DOUBLY LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.
```

Explanation:

1. A **doubly linked list** is a collection of nodes where each node contains **three parts**: data, a link to the previous node, and a link to the next node.
2. Each node has:
   - **Data** – stores the value.
   - **Previous (prev)** – points to the previous node.
   - **Next (next)** – points to the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and maintain the data, prev, and next references.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are added one by one at the end of the list.
7. For the first node, both **head and tail** point to the same node. The prev of the head and the next of the tail are None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node becomes the head, its next points to the previous head, and the previous head's prev points back to the new node.
9. During **insertion at the end**, the new node is placed after the current tail. The new node's prev points to the old tail, the old tail's next points to the new node, and the new node becomes the tail.

10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the previous and next nodes, and both prev and next links are updated.
11. During **deletion by value**, the list is searched for the node containing the given value. Once found, its previous and next nodes are connected to each other, removing the selected node from the list.
12. If the node to be deleted is the **head**, the next node becomes the new head and its prev is set to None.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head. The new head's prev is set to None.
16. During **deletion at the end**, the tail node is removed and the previous node becomes the new tail. The new tail's next is set to None.
17. The **count operation** traverses the list from the head to the tail and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves forward using the next link to display all the nodes.
19. Since it is a doubly linked list, the program can also traverse the list **backwards**, starting from the tail and using the prev links.
20. The program displays the **head and tail** values to show the starting and ending nodes of the list.
21. The important relationships maintained in a doubly linked list are:

head.prev = None

tail.next = None

22. The overall structure of a doubly linked list is:

None ← [10] ⇄ [20] ⇄ [30] ⇄ [40] → None

↑ ↑

HEAD TAIL

23. The arrows in both directions allow the list to be traversed **forward as well as backward**.

24. The program is **menu-driven**, allowing the user to perform creation, insertion, deletion, counting, displaying, and other operations repeatedly until the **Exit** option is selected.

Error table:

| Sno | Error obtained | Rectification |
|---|---|---|
| 1 | Forgetting to update the prev pointer | Update both prev and next pointers whenever a node is inserted |
| 2 | Incorrect head.prev | Maintain head.prev = None |

AIM: write a python program to implement a circular linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or travsere

x: display the head and tail

xi. exit

code:

*cll.py - C:/Users/SREEHAAS/OneDrive/Desktop/dsa lab/week4/cll.py (3.13.7)*

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # i. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for _ in range(n):
            value = int(input("Enter value: "))
            self.insert_end(value)

    # ii. Insert at beginning
    def insert_beginning(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

    # iii. Insert at end
    def insert_end(self, value):
        new_node = Node(value)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.tail.next = new_node
            self.tail = new_node
```

```python
# iv. Insert at specific index
def insert_at_index(self, value, index):
    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.insert_beginning(value)
        return

    if self.head is None:
        print("Index out of range")
        return

    current = self.head
    position = 0

    while position < index - 1 and current != self.tail:
        current = current.next
        position += 1

    if position != index - 1:
        print("Index out of range")
        return

    new_node = Node(value)
    new_node.next = current.next
    current.next = new_node

    if current == self.tail:
        self.tail = new_node
```

```python
# v. Delete by value
def delete_by_value(self, value):
    if self.head is None:
        print("List is empty")
        return

    current = self.head
    previous = self.tail

    while True:
        if current.data == value:
            break

        previous = current
        current = current.next

        if current == self.head:
            print("Value not found")
            return

    # Only one node
    if self.head == self.tail:
        self.head = None
        self.tail = None
        return

    # Delete head
    if current == self.head:
        self.head = self.head.next
        self.tail.next = self.head
        return

    # Delete other node
    previous.next = current.next

    # Delete tail
    if current == self.tail:
        self.tail = previous
```

```
# vi. Delete at beginning
def delete_beginning(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head == self.tail:
        self.head = None
        self.tail = None
    else:
        self.head = self.head.next
        self.tail.next = self.head

# vii. Delete at end
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head == self.tail:
        self.head = None
        self.tail = None
        return

    current = self.head

    while current.next != self.tail:
        current = current.next

    current.next = self.head
    self.tail = current

# viii. Count nodes
def count_nodes(self):
    if self.head is None:
        return 0

    count = 0
    current = self.head

    while True:
        count += 1
        current = current.next

        if current == self.head:
            break

    return count
```

```
    # ix. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        current = self.head

        while True:
            print(current.data, end=" -> ")
            current = current.next

            if current == self.head:
                break

        print("(Head)")

    # x. Display head and tail
    def display_head_tail(self):
        if self.head is None:
            print("List is empty")
            return

        print("Head:", self.head.data)
        print("Tail:", self.tail.data)
        print("Tail points to:", self.tail.next.data)


cll = CircularLinkedList()

while True:

    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at end")
    print("8. Count nodes")
    print("9. Display / Traverse")
    print("10. Display Head and Tail")
    print("11. Exit")

    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        cll.create()

    elif choice == 2:
        value = int(input("Enter value: "))
        cll.insert_beginning(value)

    elif choice == 3:
        value = int(input("Enter value: "))
        cll.insert_end(value)

    elif choice == 4:
        value = int(input("Enter value: "))
        index = int(input("Enter index: "))
        cll.insert_at_index(value, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        cll.delete_by_value(value)

    elif choice == 6:
        cll.delete_beginning()

    elif choice == 7:
        cll.delete_end()

    elif choice == 8:
        print("Number of nodes:", cll.count_nodes())

    elif choice == 9:
        cll.display()

    elif choice == 10:
        cll.display_head_tail()

    elif choice == 11:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

Output:

Creating a linked list:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter value: 1
Enter value: 2
Enter value: 3
```

Inserting at the beginning:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 2
Enter value: 4
```

Insert at the end:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 3
Enter value: 5
```

Insert at the specific index:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 4
Enter value: 3
Enter index: 2
```

Delete by value:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 5
Enter value to delete: 10
Value not found
```

Delete at the beginning:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 6
```

Delete at the end:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 7
```

Number of nodes:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 8
Number of nodes: 4
```

Display all the nodes:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 9
1 -> 3 -> 2 -> 3 -> (Head)
```

Display the head and the tail

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 10
Head: 1
Tail: 3
Tail points to: 1
```

Exit: the program:

```
----- CIRCULAR LINKED LIST -----
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at end
8. Count nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 11
Program exited.
```

Explanation :

1. A **circular singly linked list** is a collection of nodes where each node contains **data** and a **link to the next node**.
2. In a circular linked list, the **last node points back to the first node (head)** instead of pointing to None.
3. The program uses two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. Each new node is inserted at the end of the list.
7. If the list is empty, the first node becomes both the **head and tail**, and its next points back to itself.

8. During **insertion at the beginning**, the new node is placed before the current head. The new node becomes the head, and the tail is connected to the new head.
9. During **insertion at the end**, the new node is added after the current tail. The new node becomes the tail, and its next points to the head.
10. During **insertion at a specific index**, the list is traversed until the required position is reached, and the new node is inserted between two existing nodes.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by connecting its previous node directly to its next node.
12. If the node to be deleted is the **head**, the next node becomes the new head and the tail is connected to it.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is connected to the head.
14. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
15. During **deletion at the end**, the node before the tail is found and made the new tail.
16. The **count operation** traverses the list and counts all the nodes. Traversal stops when the head is reached again.
17. The **display/traverse operation** starts from the head and visits every node until it reaches the head again.
18. Since the list is circular, traversal does not use current != None. Instead, it stops when current becomes equal to head.
19. The program also displays the **head, tail, and tail.next** to show the circular connection.
20. The program is **menu-driven**, so the user can repeatedly perform different operations until the **Exit** option is selected.
21. The most important property maintained throughout the program is:

tail.next = head

22. Thus, the structure of the circular linked list is:

HEAD → Node → Node → Node → TAIL

↑ |

|_____|

This ensures that the linked list remains **circular after every insertion and deletion operation**.

Error table:

| sno | Error obtained | Rectification |
|---|---|---|
| 1 | Using temp != None as the traversal condition | Stop traversal when temp == head |
| 2 | Forgetting to update tail.next when the head changes | After changing the head, update tail.next = head |
| | | |

# WEEK-5:

5A.

Aim: write a python program to implement the stack using the arrays. And also implement stack operations such as push, pop, peek and also display the stack elements.

Code:

```
class Stack:
    def __init__(self, size):
        self.stack = []
        self.size = size

    # Push operation
    def push(self, value):
        if len(self.stack) == self.size:
            print("Stack Overflow")
        else:
            self.stack.append(value)
            print(value, "pushed into stack")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            print("Stack Underflow")
        else:
            value = self.stack.pop()
            print("Popped element:", value)

    # Peek operation
    def peek(self):
        if len(self.stack) == 0:
```

```
            print("Stack is empty")
        else:
            print("Top element:", self.stack[-1])

    # Display operation
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Stack elements:")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])


size = int(input("Enter stack size: "))
s = Stack(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
```

```
        s.push(value)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

Pushing an element into the stack

```
================================================== RESTART: C:\Users\Yeswanth\AppData\Local\Programs\Python\Python314\week5a.py
Enter stack size: 5

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: |
```

Popping an element from the stack

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 2
```

Peek element

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Stack is empty
```

Display all the elements in the stack

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements:
3
2
```

Exiting the program

```
----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program exited.
```

Explanation:

**1. Class and Initialization**

- A class named Stack is created.
- The __init__() method acts as the constructor.
- An empty list self.stack is created to store stack elements.
- The stack follows the **LIFO (Last In, First Out)** principle.

**2. Push Operation**

- The push() method is used to insert an element into the stack.
- The append() function adds the element to the end of the list.
- The newly inserted element becomes the **top element**.
- A message is displayed after the element is inserted.

**3. Pop Operation**

- The pop() method is used to remove the top element.
- The program first checks whether the stack is empty.
- If the stack is empty, **Stack Underflow** is displayed.
- Otherwise, the pop() function removes the last element.
- The removed element is displayed.

**4. Peek Operation**

- The peek() method is used to view the top element.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.stack[-1] is used to access the top element.
- The top element is displayed without removing it.

**5. Display Operation**

- The display() method is used to display all stack elements.
- The program first checks whether the stack is empty.
- If empty, **Stack is empty** is displayed.
- Otherwise, a for loop is used to traverse the stack.
- The elements are displayed from **top to bottom**.

**6. Count Operation**

- The count() method is used to find the number of elements in the stack.
- The len() function returns the number of elements.
- The count is displayed to the user.

Error table:

| sno | Error obtained | Error rectification |
|---|---|---|

| | | |
|---|---|---|
| 1 | Stack underflow | Check whether the stack is empty before poping |
| 2 | Name error | Define the class before creating the object |

5b.

Aim: write a python program to implement the stack using the linked list. And also implement stack operations such as push, pop, peek,count and also display the stack elements.

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None
        self.count = 0

    # Push operation
    def push(self, value):
        new_node = Node(value)

        new_node.next = self.top
        self.top = new_node
```

```
    self.count += 1

    print(value, "pushed into stack")

# Pop operation
def pop(self):
    if self.top is None:
        print("Stack Underflow")
        return

    value = self.top.data
    self.top = self.top.next
    self.count -= 1

    print("Popped element:", value)

# Peek operation
def peek(self):
    if self.top is None:
        print("Stack is empty")
    else:
        print("Top element:", self.top.data)

# Count operation
def count_nodes(self):
    print("Number of elements:", self.count)

# Display operation
def display(self):
```

```
        if self.top is None:
            print("Stack is empty")
            return

        current = self.top

        print("Stack elements:")

        while current:
            print(current.data)
            current = current.next


stack = Stack()

while True:
    print("\n----- STACK USING LINKED LIST -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Count")
    print("5. Display")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        stack.push(value)
```

```
    elif choice == 2:
        stack.pop()

    elif choice == 3:
        stack.peek()

    elif choice == 4:
        stack.count_nodes()

    elif choice == 5:
        stack.display()

    elif choice == 6:
        print("Program exited.")
        break

    else:
        print("Invalid choice")
```

Output:

Pushing an element into the
stack

```
========== RESTART: C:\Users\Yeswanth\AppData\Local\Programs\Python\Python314\week5b.py ==

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
```

Poping an element from the stack

```
----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 2
Popped element: 5
```

Peek element

```
----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 3
Top element: 3
```

Display all the elements in the stack

```
----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 5
Stack elements:
3
2
```

count

```
----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 4
Number of elements: 2
```

Exiting the program

```
----- STACK USING LINKED LIST -----
1. Push
2. Pop
3. Peek
4. Count
5. Display
6. Exit
Enter your choice: 6
Program exited.
```

Explanation:

**1. Node Class**

- A class named Node is created to represent each element of the linked list.
- The __init__() method initializes a node.
- self.data stores the value of the node.
- self.next stores the reference to the next node.
- Initially, self.next is set to None.

**2. Stack Class**

- A class named Stack is created to implement the stack operations.
- The __init__() method initializes the stack.
- self.top = None indicates that the stack is initially empty.
- The top variable stores the reference to the top node of the stack.

**3. Push Operation**

- The push() method is used to insert an element into the stack.
- A new node is created using Node(data).
- The new node is connected to the current top node using:

new_node.next = self.top

- The new node is then made the top node using:

self.top = new_node

- The inserted element is displayed.
- The operation follows the **LIFO (Last In, First Out)** principle.

**4. Pop Operation**

- The pop() method is used to remove the top element.
- The program first checks whether self.top is None.
- If self.top is None, the stack is empty and **Stack Underflow** is displayed.
- Otherwise, the data of the top node is stored in data.
- The top is moved to the next node using:

self.top = self.top.next

- The removed element is displayed.

**5. Peek Operation**

- The peek() method is used to view the top element without removing it.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.top.data is used to display the top element.
- The stack remains unchanged after the peek operation.

**6. Display Operation**

- The display() method is used to display all the elements of the stack.
- The program first checks whether the stack is empty.
- A temporary variable current is initialized with self.top.
- A while loop is used to traverse the linked list.
- current.data displays the value of each node.
- current = current.next moves to the next node.
- The elements are displayed from **top to bottom**.

**7. Count Operation**

- The count() method is used to find the number of elements in the stack.

- A variable count is initialized to 0.
- A temporary variable current starts from self.top.
- The linked list is traversed using a while loop.
- The count value is increased for every node.
- The total number of elements is displayed.

Error table:

| sno | Error obtained | Error rectification |
|---|---|---|
| 1 | Incorrect push | Set new_node.next = self.top before updating self.top |
| 2 | Infinte loop | Use break when choice == 6 |

## Week-6

Aim : Write a python program to implement a queue using an Array, Linked list, circular queue. Implement the queue operations such as enqueue, dequeue, peak and also display the queue elements.

Program: Circular Queue

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    # Enqueue operation
    def enqueue(self, value):
        if (self.rear + 1) % self.size == self.front:
            print("Circular Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
                self.rear = 0
            else:
                self.rear = (self.rear + 1) % self.size

            self.queue[self.rear] = value
            print(value, "inserted into the queue")

    # Dequeue operation
    def dequeue(self):
        if self.front == -1:
```

```
            print("Circular Queue Underflow")
        else:
            value = self.queue[self.front]
            self.queue[self.front] = None

            if self.front == self.rear:
                self.front = -1
                self.rear = -1
            else:
                self.front = (self.front + 1) % self.size

            print(value, "deleted from the queue")

    # Peek operation
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front

            while True:
```

```
            print(self.queue[i])
            if i == self.rear:
                break
            i = (i + 1) % self.size


size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)

while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Peek")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter the element to enqueue: "))
        q.enqueue(value)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.display()
```

```
    elif choice == 4:
        q.peek()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice")
```

Output:

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week6a.py ==========
Enter the size of the queue: 3

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Peek
5. Exit
Enter your choice: 1
Enter the element to enqueue: 2
2 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Peek
5. Exit
Enter your choice: 3
The elements of the queue are:
2

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Peek
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

**Queue Using Array**

**Structure:**

- Uses a Python list self.queue to store elements.
- self.front keeps track of the **first element** in the queue.
- self.rear keeps track of the **last element** in the queue.
- self.size defines the **maximum capacity** of the queue.
- Initially:

- front = -1
- rear = -1

**Operations:**

**1. Enqueue**

- Check if rear == size - 1 → **Overflow** (queue is full).
- If the queue is empty, set front = 0.
- Increment rear.
- Insert the element at rear.

**2. Dequeue**

- Check if front == -1 or front > rear → **Underflow** (queue is empty).
- Remove the element at front.
- Increment front.
- If the queue becomes empty, reset front and rear to -1.

**3. Peek**

- Shows the element at front without removing it.
- If the queue is empty → Underflow/empty message.

**4. Display**

- Prints elements from front to rear.
- Elements are displayed in **front-to-rear order**.

**Important Points:**

- Queue follows **FIFO (First In, First Out)**.
- Insertion takes place at the **rear**.
- Deletion takes place at the **front**.
- **Overflow** → queue is full.
- **Underflow** → queue is empty.

A0B

Error table:

**ERROR TABLE:**

| S.NO | Error Obtained | Rectification |
|---|---|---|

| | | |
|---|---|---|
| 1. | Display on empty circular queue. | Add condition: if self.front == -1: → print "Queue is empty". |
| 2. | Circular Queue Overflow | Add a check: if (self.rear + 1) % self.size == self.front: → print "Circular Queue Overflow". |
| 3. | Circular Queue Underflow | Add a check: if self.front == -1: → print "Circular Queue Underflow". |
| 4. | rear does not wrap around to the beginning of the array. | Update using: self.rear = (self.rear + 1) % self.size. |
| | | |

Program: Array

Code:

```
class Queue:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    # Enqueue operation
    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
```

```
        self.queue[self.rear] = x
        print(x, "inserted into the queue")

# Dequeue operation
def dequeue(self):
    if self.front == -1 or self.front > self.rear:
        print("Queue Underflow")
    else:
        x = self.queue[self.front]
        self.queue[self.front] = None
        self.front += 1

        print(x, "deleted from the queue")

        if self.front > self.rear:
            self.front = -1
            self.rear = -1

# Peek operation
def peek(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        print("Front element:", self.queue[self.front])

# Display operation
def display(self):
    if self.front == -1:
        print("Queue is empty")
```

```
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of queue: "))
q = Queue(size)

while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()
```

```
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid choice")
```

Output:

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week6b.py ==========
Enter the size of queue: 3

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Queue Underflow

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 5
5 inserted into the queue
```

Explanation:

**Queue Using Linked List**

**Structure:**

- Each element is represented using a **Node**.
- Each Node contains:
    - data → stores the value.
    - next → points to the next node.
- self.front points to the **first node**.
- self.rear points to the **last node**.
- Initially:
    - front = None
    - rear = None

**Operations:**

**1. Enqueue**

- Create a new node.
- If the queue is empty:
    - Set both front and rear to the new node.
- Otherwise:
    - Link rear.next to the new node.
    - Update rear to the new node.

**2. Dequeue**

- Check if front is None → **Underflow**.
- Store/remove the node at front.
- Move front to front.next.
- If front becomes None, set rear = None.

**3. Peek**

- Shows the data stored at front without removing it.
- If front is None, the queue is empty.

**4. Display**

- Start from front.
- Traverse using next.
- Print each element until None.

**Important Points:**

- Follows **FIFO**.
- Insertion → **rear**.
- Deletion → **front**.
- No fixed size is required.
- Memory is allocated dynamically.

**ERROR TABLE: QUEUE USING LINKED LIST**

| S.NO | Error Obtained | Rectification |
|---|---|---|
| 1. | Display on empty queue. | Add condition: if self.front is None: → print "Queue is empty". |

| | | |
|---|---|---|
| 2. | Queue Underflow when deleting from an empty queue. | Add a check: if self.front is None: → print "Queue Underflow". |
| 3. | First node inserted without updating front and rear. | If the queue is empty, set self.front = self.rear = new. |
| 4. | rear not updated after insertion. | Update using self.rear = new after linking the new node. |

Program: Linked List

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, value):
        new_node = Node(value)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
```

```
    print(value, "inserted into the queue")

# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue Underflow")
    else:
        value = self.front.data
        self.front = self.front.next

        if self.front is None:
            self.rear = None

        print(value, "deleted from the queue")

# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("Front element:", self.front.data)

# Display operation
def display(self):
    if self.front is None:
        print("Queue is empty")
    else:
        current = self.front
```

```
            print("Queue elements:")

            while current:
                print(current.data)
                current = current.next


q = Queue()

while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()
```

```
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break
    else:
        print("Invalid choice")
```

Output:

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week6c.py ==========

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 5
5 inserted into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 3
3 inserted into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 5

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
```

Explanation:

**Circular Queue Using Array**

**Structure:**

- Uses a Python list self.queue to store elements.
- self.front points to the **first element**.
- self.rear points to the **last element**.
- self.size defines the maximum capacity.

- When rear reaches the last index, it can **wrap around to index 0**.
- This allows previously freed positions to be reused.

**Operations:**

**1. Enqueue**

- Check if (rear + 1) % size == front → **Overflow** (circular queue is full).
- If the queue is empty:
    - Set front = 0
    - Set rear = 0
- Otherwise:
    - Move rear using:
      rear = (rear + 1) % size
- Insert the element at rear.

**2. Dequeue**

- Check if front == -1 → **Underflow**.
- Remove the element at front.
- If front == rear:
    - The queue becomes empty.
    - Set front = -1 and rear = -1.
- Otherwise:
    - Move front using:
      front = (front + 1) % size

**3. Peek**

- Shows the element at front without removing it.
- If front == -1, the queue is empty.

**4. Display**

- Start from front.
- Move towards rear.
- Use modulo % size to wrap around to the beginning of the array.

**Important Points:**

- Follows **FIFO**.

- Insertion → **rear**.
- Deletion → **front**.
- The last position connects logically back to the first position.
- **Unused spaces can be reused**, unlike a simple linear array queue.
- Overflow occurs when the next rear position becomes the front.

Error Table: **QUEUE USING ARRAY**

| S.NO | Error Obtained | Rectification |
|---|---|---|
| 1. | Display on empty queue. | Add condition: if self.front == -1: → print "Queue is empty". |
| 2. | Queue Overflow when inserting beyond the defined size. | Add a check: if self.rear == self.size - 1: → print "Queue Overflow". |
| 3. | Queue Underflow when deleting from an empty queue. | Add a check: if self.front == -1 or self.front > self.rear: → print "Queue Underflow". |
| 4. | First element inserted without setting front. | If the queue is empty, set self.front = 0 before increasing rear. |

# WEEK-7:

AIM a)write a python program to implement binary search tree using an array with all its operations.

Program:

```
class BST:


    def __init__(self):
        self.tree = [None] * 100


    def insert(self, data):
        i = 0


        while i < 100:
            if self.tree[i] is None:
                self.tree[i] = data
                print("Element inserted")
                return


            if data < self.tree[i]:
                i = 2 * i + 1
            elif data > self.tree[i]:
                i = 2 * i + 2
            else:
                print("Duplicate elements not allowed")
                return


        print("Tree is full")


    def search(self, data):
        i = 0
```

```
    while i < 100 and self.tree[i] is not None:
        if self.tree[i] == data:
            return True

        if data < self.tree[i]:
            i = 2 * i + 1
        else:
            i = 2 * i + 2

    return False

def delete(self, data):
    if not self.search(data):
        print("Element not found")
        return

    values = []

    def collect(i):
        if i >= 100 or self.tree[i] is None:
            return

        collect(2 * i + 1)

        if self.tree[i] != data:
            values.append(self.tree[i])

        collect(2 * i + 2)
```

```
    collect(0)

    self.tree = [None] * 100

    for value in values:
        self.insert(value)

    print("Element deleted")

def inorder(self, i):
    if i >= 100 or self.tree[i] is None:
        return

    self.inorder(2 * i + 1)
    print(self.tree[i], end=" ")
    self.inorder(2 * i + 2)

def preorder(self, i):
    if i >= 100 or self.tree[i] is None:
        return

    print(self.tree[i], end=" ")
    self.preorder(2 * i + 1)
    self.preorder(2 * i + 2)

def postorder(self, i):
    if i >= 100 or self.tree[i] is None:
        return
```

```
        self.postorder(2 * i + 1)
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")

    def minimum(self):
        if self.tree[0] is None:
            print("Tree is empty")
            return

        i = 0

        while 2 * i + 1 < 100 and self.tree[2 * i + 1] is not None:
            i = 2 * i + 1

        print("Minimum:", self.tree[i])

    def count(self, i):
        if i >= 100 or self.tree[i] is None:
            return 0

        return 1 + self.count(2 * i + 1) + self.count(2 * i + 2)

    def height(self, i):
        if i >= 100 or self.tree[i] is None:
            return 0

        left = self.height(2 * i + 1)
        right = self.height(2 * i + 2)
```

```
        return 1 + max(left, right)


# Main Program

bst = BST()

while True:
    print("\n----- BST MENU -----")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Find Minimum")
    print("8. Count Nodes")
    print("9. Find Height")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        bst.insert(data)

    elif choice == 2:
        data = int(input("Enter element to delete: "))
```

```
        bst.delete(data)

    elif choice == 3:
        data = int(input("Enter element to search: "))

        if bst.search(data):
            print("Element found")
        else:
            print("Element not found")

    elif choice == 4:
        print("Inorder:")
        bst.inorder(0)
        print()

    elif choice == 5:
        print("Preorder:")
        bst.preorder(0)
        print()

    elif choice == 6:
        print("Postorder:")
        bst.postorder(0)
        print()

    elif choice == 7:
        bst.minimum()

    elif choice == 8:
```

```
    print("Total Nodes:", bst.count(0))

elif choice == 9:
    print("Height:", bst.height(0))

elif choice == 10:
    print("Exiting program...")
    break

else:
    print("Invalid choice")
```

output:

```
================================================== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python,

----- BST MENU -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Count Nodes
9. Find Height
10. Exit
Enter your choice: 1
Enter element: 5
Element inserted

----- BST MENU -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Count Nodes
9. Find Height
10. Exit
Enter your choice: 5
Preorder:
5

----- BST MENU -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Count Nodes
9. Find Height
10. Exit
Enter your choice: 4
```

Explanation:

**Binary Search Tree (BST) Using Array**

**Structure:**

- Uses a Python list self.tree to store the elements.
- The root element is stored at **index 0**.
- For a node at index i:
    - **Left child** → 2*i + 1
    - **Right child** → 2*i + 2
- An empty position is represented using None.
- BST follows the rule:
    - Left subtree contains **smaller** elements.
    - Right subtree contains **greater** elements.

**Operations:**

**1. Insert**

- Start from index 0 (root).
- Compare the new element with the current element.
- If the element is smaller → move to 2*i + 1.
- If the element is greater → move to 2*i + 2.
- Continue until an empty position (None) is found.
- Insert the element at that position.

**2. Search**

- Start from the root.
- Compare the required element with the current element.
- If equal → element is found.
- If smaller → move to the left child.
- If greater → move to the right child.
- If None is reached → element is not present.

**3. Delete**

There are three cases:

- **No child:** Set that position to None.
- **One child:** Replace the node with its child.
- **Two children:** Find the minimum element in the right subtree (inorder successor), replace the deleted element with it, and remove the original successor.

**4. Minimum**

- Start from the given node.
- Keep moving to the left child.
- The leftmost element is the minimum value.

**5. Maximum**

- Start from the given node.
- Keep moving to the right child.
- The rightmost element is the maximum value.

**6. Inorder Traversal**

- Traverse **Left → Root → Right**.
- Inorder traversal of a BST gives elements in **sorted order**.

**7. Preorder Traversal**

- Traverse **Root → Left → Right**.

**8. Postorder Traversal**

- Traverse **Left → Right → Root**.

**9. Count**

- Counts the total number of nodes in the BST.
- Formula:
  **Count = 1 + left subtree count + right subtree count**

**10. Height**

- Finds the longest path from the root to a leaf.
- Formula:
  **Height = 1 + max(left height, right height)**

**Important Points:**

- Root is at index 0.
- Left child → 2*i + 1.
- Right child → 2*i + 2.
- None represents an empty position.
- Inorder traversal gives **sorted elements**.
- BST provides efficient searching when the tree is balanced.

Error Table:

| S.NO | Error Obtained | Rectification |
|---|---|---|
| 1. | Inserting into an occupied position without comparing the value. | Compare data with self.tree[i]. If data is smaller, move to 2*i + 1; otherwise move to 2*i + 2. |
| 2. | Searching beyond the array limit. | Check the index condition: while i < 100 and self.tree[i] is not None. |

| | | |
|---|---|---|
| 3. | Searching for a value that does not exist. | If self.tree[i] is None, return False. |
| 4. | Incorrect child index calculation. | Use left child = 2*i + 1 and right child = 2*i + 2. |

AIM b) write a python program to implement binary tree using an linked list with all its operations.

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BST:

    def __init__(self):
        self.root = None

    # Insert operation
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
```

```
    elif data > root.data:
        root.right = self.insert(root.right, data)

    else:
        print("Duplicate elements not allowed")

    return root

# Search operation
def search(self, root, data):
    if root is None:
        return False

    if root.data == data:
        return True

    if data < root.data:
        return self.search(root.left, data)

    return self.search(root.right, data)

# Find minimum node
def find_min(self, root):
    current = root

    while current.left is not None:
        current = current.left

    return current
```

```
# Delete operation
def delete(self, root, data):
    if root is None:
        return root

    if data < root.data:
        root.left = self.delete(root.left, data)

    elif data > root.data:
        root.right = self.delete(root.right, data)

    else:
        if root.left is None:
            return root.right

        elif root.right is None:
            return root.left

        temp = self.find_min(root.right)
        root.data = temp.data
        root.right = self.delete(root.right, temp.data)

    return root

# Inorder traversal
def inorder(self, root):
    if root:
        self.inorder(root.left)
```

```
        print(root.data, end=" ")
        self.inorder(root.right)

# Preorder traversal
def preorder(self, root):
    if root:
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

# Postorder traversal
def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")

# Minimum operation
def minimum(self):
    if self.root is None:
        print("Tree is empty")
        return

    temp = self.find_min(self.root)
    print("Minimum:", temp.data)

# Maximum operation
def maximum(self):
    if self.root is None:
```

```
            print("Tree is empty")
            return

        temp = self.root

        while temp.right is not None:
            temp = temp.right

        print("Maximum:", temp.data)

    # Count operation
    def count(self, root):
        if root is None:
            return 0

        return 1 + self.count(root.left) + self.count(root.right)

    # Height operation
    def height(self, root):
        if root is None:
            return 0

        left = self.height(root.left)
        right = self.height(root.right)

        return 1 + max(left, right)


# Main Program
```

```
bst = BST()

while True:
    print("\n----- BST MENU -----")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Find Minimum")
    print("8. Count Nodes")
    print("9. Find Height")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter element: "))
        bst.root = bst.insert(bst.root, data)

    elif choice == 2:
        data = int(input("Enter element to delete: "))

        if bst.search(bst.root, data):
            bst.root = bst.delete(bst.root, data)
            print("Element deleted")
        else:
```

```
            print("Element not found")

    elif choice == 3:
        data = int(input("Enter element to search: "))

        if bst.search(bst.root, data):
            print("Element found")
        else:
            print("Element not found")

    elif choice == 4:
        print("Inorder:")
        bst.inorder(bst.root)
        print()

    elif choice == 5:
        print("Preorder:")
        bst.preorder(bst.root)
        print()

    elif choice == 6:
        print("Postorder:")
        bst.postorder(bst.root)
        print()

    elif choice == 7:
        bst.minimum()

    elif choice == 8:
```

```
        print("Total Nodes:", bst.count(bst.root))

    elif choice == 9:
        print("Height:", bst.height(bst.root))

    elif choice == 10:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

Output:

```
----- BST MENU -----
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Find Minimum
8. Count Nodes
9. Find Height
10. Exit
Enter your choice: 4
Inorder:
40 50 70
```

Explanation:

**Binary Search Tree (BST) Using Linked List**

**Structure:**

- Each element is represented using a **Node**.
- Each Node contains:
    - data → stores the value.

    - left → points to the left child.
    - right → points to the right child.
- self.root points to the **root node** of the BST.
- Initially, root = None, which means the tree is empty.
- BST follows the rule:
    - Left subtree contains **smaller** elements.
    - Right subtree contains **greater** elements.

**Operations:**

**1. Insert**

- Create a new node.
- If the tree is empty, make the new node the root.
- Otherwise, start from the root.
- If the new element is smaller → move to the left.
- If the new element is greater → move to the right.
- Continue until an empty position is found.
- Link the new node at that position.

**2. Search**

- Start from the root.
- Compare the required element with the current node.
- If equal → element is found.
- If smaller → move to the left child.
- If greater → move to the right child.
- If None is reached → element is not present.

**3. Delete**

There are three cases:

- **No child:** Remove the node.
- **One child:** Replace the node with its child.
- **Two children:** Find the minimum element in the right subtree (inorder successor), copy its value into the node, and delete the original successor node.

**4. Minimum**

- Start from the given node.
- Keep moving to the left child.
- The leftmost node contains the minimum value.

**5. Maximum**

- Start from the given node.
- Keep moving to the right child.
- The rightmost node contains the maximum value.

**6. Inorder Traversal**

- Traverse **Left → Root → Right**.
- Gives BST elements in **sorted order**.

**7. Preorder Traversal**

- Traverse **Root → Left → Right**.

**8. Postorder Traversal**

- Traverse **Left → Right → Root**.

**9. Count**

- Counts the total number of nodes.
- Formula:
  **Count = 1 + left subtree count + right subtree count**

**10. Height**

- Finds the longest path from the root to a leaf.
- Formula:
  **Height = 1 + max(left height, right height)**

**Important Points:**

- root points to the first node.
- left stores the reference to the left child.
- right stores the reference to the right child.
- None represents an empty child.
- Inorder traversal gives **sorted elements**.
- Linked-list representation uses **dynamic memory**.

**Error table:**

| S.NO | Error Obtained | Rectification |
|---|---|---|
| 1. | Inserting the first node without setting root. | If self.root is None, set self.root = new and return. |
| 2. | Searching for a value that does not exist. | Continue while temp is not None; if temp becomes None, return False. |
| 3. | Incorrect movement while searching. | If data < temp.data, move to temp.left; otherwise move to temp.right. |
| 4. | Deleting a node with two children incorrectly. | Find the minimum value in the right subtree, replace the node data, and delete the successor. |

week 8

Aim : write a python program to implement Binary tree with its operations using array and linked list.

**Program: using array**

```
class binarytree:

    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    def insert_root(self, data):
        self.tree[0] = data

    def set_right(self, i, data):
        child = 2 * i + 2

        if child < self.size:
            self.tree[child] = data
        else:
            print("Index out of range")
```

```
def set_left(self, i, data):
    child = 2 * i + 1

    if child < self.size:
        self.tree[child] = data
    else:
        print("Index out of range")

def preorder(self, i=0):
    if i >= self.size or self.tree[i] is None:
        return

    print(self.tree[i], end=" ")
    self.preorder(2 * i + 1)
    self.preorder(2 * i + 2)

def inorder(self, i=0):
    if i >= self.size or self.tree[i] is None:
        return

    self.inorder(2 * i + 1)
    print(self.tree[i], end=" ")
    self.inorder(2 * i + 2)

def postorder(self, i=0):
    if i >= self.size or self.tree[i] is None:
        return

    self.postorder(2 * i + 1)
```

```
        self.postorder(2 * i + 2)
        print(self.tree[i], end=" ")

    def levelorder(self):
        for value in self.tree:
            if value is not None:
                print(value, end=" ")

    def display(self):
        if self.tree[0] is None:
            print("No elements in tree")
            return

        for i in range(self.size):
            print(f"Index {i}: {self.tree[i]}")



# Main

tree = binarytree(7)

tree.insert_root('A')
tree.set_left(0, 'B')
tree.set_right(0, 'C')

print("\npreorder\n", end=" ")
tree.preorder()

print("\ninorder\n", end=" ")
```

```
tree.inorder()

print("\npostorder\n", end=" ")
tree.postorder()

print("\nlevel order\n", end=" ")
tree.levelorder()

print("\ndisplay\n", end=" ")
tree.display()
```

**Output:**

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week8a.py =

preorder
 A B C
inorder
 B A C
postorder
 B C A
level order
 A B C
display
 Index 0: A
Index 1: B
Index 2: C
```

**Explanation:**

- The binarytree class is used to implement a binary tree using an array. The tree list is initialized with None values according to the given size.
- The insert_root() function stores the root node at index 0 because the root is always the first element in the array representation.
- The set_left() function is used to insert the left child of a node. For a node at index i, its left child is stored at index **2*i + 1**.
- The set_right() function is used to insert the right child of a node. For a node at index i, its right child is stored at index **2*i + 2**.
- The if child < self.size condition checks whether the calculated child index is within the available array size. If it exceeds the size, an index-out-of-range message is displayed.

- The preorder() function visits the current node first, followed by its left subtree and right subtree. The child indexes are calculated using 2*index + 1 and 2*index + 2.
- The inorder() function first visits the left subtree, then the current node and finally the right subtree.
- The postorder() function first visits the left subtree, then the right subtree and finally the current node.
- The levelorder() function visits the elements from the beginning to the end of the array. Since the array stores the binary tree level by level, this directly gives the level-order traversal.
- The display() function uses a loop to display every index of the array along with the value stored at that index.
- The condition self.tree[index] is None is used during traversal to stop when an empty position is reached.

**Error Table:**

| S.No | Common Error | Correction |
|---|---|---|
| 1 | Not setting left and right references to None while creating a node. | Initialize both left and right as None in the Node constructor. |
| 2 | Calling traversal functions without passing the current node correctly. | Pass the current node during recursive calls. |
| 3 | Forgetting the base condition when the current node is None. | Use a condition such as if root is None: return. |
| 4 | Using the wrong traversal order. | Preorder: Root-Left-Right, Inorder: Left-Root-Right, |

| | | |
|---|---|---|
| | | Postorder: Left-Right-Root. |

**Program: using linked list**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class binarytree:

    def __init__(self):
        self.root = None

    def create(self):
        self.root = Node('A')
        self.root.left = Node('B')
        self.root.right = Node('C')
        self.root.left.left = Node('D')
        self.root.left.right = Node('E')

    def preorder(self, node):
        if node is None:
            return

        print(node.data, end=" ")
        self.preorder(node.left)
```

```
        self.preorder(node.right)

    def inorder(self, node):
        if node is None:
            return

        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)

    def postorder(self, node):
        if node is None:
            return

        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data, end=" ")

    def levelorder(self):
        if self.root is None:
            return

        queue = [self.root]

        while queue:
            node = queue.pop(0)
            print(node.data, end=" ")

            if node.left:
```

```
            queue.append(node.left)

        if node.right:
            queue.append(node.right)


# Main

tree = binarytree()

tree.create()

print("\npreorder\n", end=" ")
tree.preorder(tree.root)

print("\ninorder\n", end=" ")
tree.inorder(tree.root)

print("\npostorder\n", end=" ")
tree.postorder(tree.root)

print("\nlevel order\n", end=" ")
tree.levelorder()
```

**Output:**

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/

preorder
 A B D E C
inorder
 D B E A C
postorder
 D E B C A
level order
 A B C D E
```

**Explanation:**

- The Node class is used to create each node of the binary tree. Each node contains three parts: data, left, and right. The data stores the value, while left and right store references to the left and right child nodes.
- The __init__() method initializes the node with the given data. Initially, both left and right are set to None because the node does not have any children.
- The root represents the first node of the binary tree. New nodes are connected to the root using the left and right references.
- The preorder() function performs preorder traversal. It first visits the root node, then recursively visits the left subtree and finally the right subtree. The order followed is **Root → Left → Right**.
- The inorder() function performs inorder traversal. It first visits the left subtree, then the root node and finally the right subtree. The order followed is **Left → Root → Right**.
- The postorder() function performs postorder traversal. It first visits the left subtree, then the right subtree and finally the root node. The order followed is **Left → Right → Root**.
- The condition if root: checks whether the current node exists. If the node is None, the recursive function stops for that particular branch.

**Error Table:**

| S.No | Common Error | Correction |
|---|---|---|
| 1 | Not setting left and right references to None while creating a node. | Initialize both left and right as None in the Node constructor. |
| 2 | Calling traversal functions without passing the current node correctly. | Pass the current node during recursive calls. |
| 3 | Forgetting the base condition when the current node is None. | Use a condition such as if root is None: return. |

| | | |
|---|---|---|
| 4 | Using the wrong traversal order. | Preorder: Root-Left-Right, Inorder: Left-Root-Right, Postorder: Left-Right-Root. |

## Week 9

**Aim :** write a python code for AVL tree using array and linked list with all its operations.

**Program:**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1


class AVLTree:

    def height(self, root):
        if root is None:
            return 0
        return root.height

    def get_balance(self, root):
        if root is None:
            return 0

        return self.height(root.left) - self.height(root.right)

    def update_height(self, root):
```

```
        root.height = 1 + max(
            self.height(root.left),
            self.height(root.right)
        )

    # Right Rotation
    def right_rotate(self, root):
        new_root = root.left
        temp = new_root.right

        new_root.right = root
        root.left = temp

        self.update_height(root)
        self.update_height(new_root)

        return new_root

    # Left Rotation
    def left_rotate(self, root):
        new_root = root.right
        temp = new_root.left

        new_root.left = root
        root.right = temp

        self.update_height(root)
        self.update_height(new_root)
```

```
        return new_root

    # Insert
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)

        elif data > root.data:
            root.right = self.insert(root.right, data)

        else:
            return root

        self.update_height(root)

        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and data < root.left.data:
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and data > root.right.data:
            return self.left_rotate(root)

        # LR Case
```

```
        if balance > 1 and data > root.left.data:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RL Case
        if balance < -1 and data < root.right.data:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Find minimum node
    def min_value_node(self, root):
        current = root

        while current.left is not None:
            current = current.left

        return current

    # Delete
    def delete(self, root, data):
        if root is None:
            return root

        if data < root.data:
            root.left = self.delete(root.left, data)

        elif data > root.data:
```

```
            root.right = self.delete(root.right, data)

        else:
            # No child
            if root.left is None and root.right is None:
                return None

            # Only right child
            elif root.left is None:
                return root.right

            # Only left child
            elif root.right is None:
                return root.left

            # Two children
            else:
                temp = self.min_value_node(root.right)
                root.data = temp.data
                root.right = self.delete(root.right, temp.data)

        self.update_height(root)

        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)
```

```
        # LR Case
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Search
    def search(self, root, data):
        if root is None:
            return False

        if root.data == data:
            return True

        if data < root.data:
            return self.search(root.left, data)

        return self.search(root.right, data)
```

```
# Inorder
def inorder(self, root):
    if root is not None:
        self.inorder(root.left)
        print(root.data, end=" ")
        self.inorder(root.right)

# Preorder
def preorder(self, root):
    if root is not None:
        print(root.data, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

# Postorder
def postorder(self, root):
    if root is not None:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.data, end=" ")

# Display tree
def display(self, root, level=0):
    if root is not None:
        self.display(root.right, level + 1)
        print("    " * level + str(root.data))
        self.display(root.left, level + 1)
```

```
# Main Program

avl = AVLTree()
root = None

while True:
    print("\n----- AVL TREE USING LINKED LIST -----")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        data = int(input("Enter value: "))
        root = avl.insert(root, data)

    elif choice == 2:
        data = int(input("Enter value: "))

        if avl.search(root, data):
            root = avl.delete(root, data)
            print("Element deleted")
        else:
```

```
            print("Element not found")

    elif choice == 3:
        data = int(input("Enter value: "))

        if avl.search(root, data):
            print("Element found")
        else:
            print("Element not found")

    elif choice == 4:
        print("Inorder:", end=" ")
        avl.inorder(root)
        print()

    elif choice == 5:
        print("Preorder:", end=" ")
        avl.preorder(root)
        print()

    elif choice == 6:
        print("Postorder:", end=" ")
        avl.postorder(root)
        print()

    elif choice == 7:
        print("AVL Tree:")
        avl.display(root)
```

```
    elif choice == 8:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

Output:

```
========================================================= RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Pytho

----- AVL TREE USING LINKED LIST -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value: 2

----- AVL TREE USING LINKED LIST -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value: 5

----- AVL TREE USING LINKED LIST -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 3
Enter value: 5
Element found
```

Explanation:

- An **AVL tree** is a self-balancing Binary Search Tree (BST).
- In an AVL tree, the difference between the height of the left and right subtrees is called the **Balance Factor**.
- The Balance Factor of every node must be **-1, 0, or +1**.
- In the array implementation, the tree nodes are stored inside a fixed-size array.
- For a node at index i:
    - Left child is stored at 2*i + 1.
    - Right child is stored at 2*i + 2.
- None is used to represent an empty position in the array.

**Main Functions**

- **__init__()**
    - Creates the array for storing the tree.
    - The size of the array is given by the user/program.
    - Initially, all positions contain None.
- **height()**
    - Calculates the height of a node.
    - It recursively checks the left and right subtrees.
    - The greater height is selected and 1 is added.
- **balance_factor()**
    - Calculates the balance factor of a node.
    - It is calculated as:
        - Height of Left Subtree - Height of Right Subtree
    - A value greater than 1 indicates that the tree is left-heavy.
    - A value less than -1 indicates that the tree is right-heavy.
- **insert()**
    - Adds a new element to the AVL tree.

    - The element is placed according to Binary Search Tree rules.
    - Smaller values are placed in the left subtree.
    - Larger values are placed in the right subtree.
    - After insertion, the tree is checked and rotations are performed if required.
- **search()**
    - Searches for a particular element.
    - If the element is found, it returns True.
    - If the element does not exist, it returns False.
- **delete()**
    - Removes an element from the tree.
    - It handles nodes with no child, one child, or two children.
    - After deletion, the tree is rebalanced.
- **find_min()**
    - Finds the smallest element in a subtree.
    - It repeatedly moves to the left child.
    - It is mainly used when deleting a node having two children.
- **rotate_left()**
    - Performs a left rotation when the tree becomes right-heavy.
    - It helps restore the AVL property.
- **rotate_right()**
    - Performs a right rotation when the tree becomes left-heavy.
    - It helps restore the AVL property.
- **balance_tree()**
    - Checks the balance factor of the nodes.
    - If the tree becomes unbalanced, suitable rotations are performed.
- **inorder()**
    - Traverses the tree in:
        - Left → Root → Right
    - It displays the elements in sorted order.

- **preorder()**
    - Traverses the tree in:
        - Root → Left → Right
- **postorder()**
    - Traverses the tree in:
        - Left → Right → Root
- **display()**
    - Displays the elements along with their array indexes.
    - It helps to understand how the tree is stored in the array.

**Main Program**

- The main program provides a **menu-driven interface**.
- The user can choose:
    - Insert
    - Delete
    - Search
    - Inorder
    - Preorder
    - Postorder
    - Display
    - Exit
- This allows all major AVL tree operations to be performed interactively.

**2. AVL Tree Using Linked List – Explanation**

- In the linked-list implementation, every tree element is represented using a **Node**.
- Each node contains:
    - Data
    - Left child
    - Right child
    - Height

- The left and right pointers connect the nodes and form the tree.
- Unlike the array implementation, memory is allocated dynamically as new nodes are created.

**Node Class**

- The Node class represents one node of the AVL tree.
- data stores the value.
- left stores the reference to the left child.
- right stores the reference to the right child.
- height stores the height of the node.
- Initially, a new node has height 1.

**Main Functions**

- **height()**
    - Returns the height of a node.
    - If the node is None, its height is considered 0.
- **get_balance()**
    - Calculates the balance factor.
    - Formula:
        - Height(Left) - Height(Right)
    - It is used to determine whether rotations are required.
- **update_height()**
    - Updates the height of a node after insertion, deletion, or rotation.
    - It takes the maximum height of the left and right subtrees and adds 1.
- **right_rotate()**
    - Performs a right rotation.
    - It is mainly used for the **LL imbalance**.
    - The left child becomes the new root of that subtree.
- **left_rotate()**
    - Performs a left rotation.
    - It is mainly used for the **RR imbalance**.

    - The right child becomes the new root of that subtree.
- **insert()**
    - Inserts a new value according to BST rules.
    - Smaller values go to the left.
    - Larger values go to the right.
    - The height of each affected node is updated.
    - The balance factor is checked.
    - If the tree becomes unbalanced, the appropriate rotation is performed.

**Four Cases During Insertion**

- **LL Case**
    - New node is inserted into the left subtree of the left child.
    - A **right rotation** is performed.
- **RR Case**
    - New node is inserted into the right subtree of the right child.
    - A **left rotation** is performed.
- **LR Case**
    - New node is inserted into the right subtree of the left child.
    - First, a **left rotation** is performed on the left child.
    - Then, a **right rotation** is performed.
- **RL Case**
    - New node is inserted into the left subtree of the right child.
    - First, a **right rotation** is performed on the right child.
    - Then, a **left rotation** is performed.
- **min_value_node()**
    - Finds the node with the minimum value.
    - It keeps moving to the left until the left child is None.
    - It is used during deletion of a node with two children.
- **delete()**
    - Deletes a node from the AVL tree.

    - Three situations are handled:
        - Node has no child.
        - Node has one child.
        - Node has two children.
    - For two children, the inorder successor is used.
    - After deletion, heights are updated.
    - The balance factor is checked and rotations are performed if required.
- **search()**
    - Searches for a given value.
    - If the value matches the current node, it returns True.
    - If the value is smaller, it searches the left subtree.
    - If the value is larger, it searches the right subtree.
- **inorder()**
    - Traverses:
        - Left → Root → Right
    - Produces the elements in sorted order.
- **preorder()**
    - Traverses:
        - Root → Left → Right
- **postorder()**
    - Traverses:
        - Left → Right → Root
- **display()**
    - Displays the tree structure vertically.
    - It helps visualize the relationship between parent and child nodes.

**Main Program**

- The main program creates an AVL tree object.
- root = None represents an initially empty tree.
- A menu is provided to perform different operations.

- The user can perform:
    - Insert
    - Delete
    - Search
    - Inorder
    - Preorder
    - Postorder
    - Display
    - Exit

Error Table:

| S.No. | Common Error | Reason / Correction |
|---|---|---|
| 1 | AttributeError: NoneType | The code tries to access a property of an empty node. Check whether the node is None before accessing its data or children. |
| 2 | Incorrect rotation | The left/right child references are changed incorrectly. Verify the temporary node and child links during rotation. |
| 3 | Wrong height or balance factor | The height is not updated after insertion, deletion, or rotation. Call the height update function at the correct place. |
| 4 | Element not found during deletion/search | The value does not exist in the tree or the search path is incorrect. Follow BST comparison rules correctly. |

Program:

```
class AVLTree:


    def __init__(self, size):
        self.size = size
        self.data = [None] * size
        self.left = [-1] * size
        self.right = [-1] * size
```

```
    self.height = [0] * size
    self.root = -1
    self.count = 0

# Create new node
def create_node(self, data):
    if self.count == self.size:
        print("Tree is full")
        return -1

    index = self.count
    self.data[index] = data
    self.left[index] = -1
    self.right[index] = -1
    self.height[index] = 1

    self.count += 1
    return index

# Get height
def get_height(self, index):
    if index == -1:
        return 0

    return self.height[index]

# Get balance factor
def get_balance(self, index):
    if index == -1:
```

```
        return 0

    return (self.get_height(self.left[index]) -
            self.get_height(self.right[index]))

# Update height
def update_height(self, index):
    self.height[index] = 1 + max(
        self.get_height(self.left[index]),
        self.get_height(self.right[index])
    )

# Right Rotation
def right_rotate(self, index):
    new_root = self.left[index]
    temp = self.right[new_root]

    self.right[new_root] = index
    self.left[index] = temp

    self.update_height(index)
    self.update_height(new_root)

    return new_root

# Left Rotation
def left_rotate(self, index):
    new_root = self.right[index]
    temp = self.left[new_root]
```

```
        self.left[new_root] = index
        self.right[index] = temp

        self.update_height(index)
        self.update_height(new_root)

        return new_root

    # Insert node
    def insert_node(self, index, data):
        if index == -1:
            return self.create_node(data)

        if data < self.data[index]:
            self.left[index] = self.insert_node(
                self.left[index], data
            )

        elif data > self.data[index]:
            self.right[index] = self.insert_node(
                self.right[index], data
            )

        else:
            print("Duplicate value")
            return index

        self.update_height(index)
```

```
        balance = self.get_balance(index)

        # LL Case
        if balance > 1 and data < self.data[self.left[index]]:
            return self.right_rotate(index)

        # RR Case
        if balance < -1 and data > self.data[self.right[index]]:
            return self.left_rotate(index)

        # LR Case
        if balance > 1 and data > self.data[self.left[index]]:
            self.left[index] = self.left_rotate(self.left[index])
            return self.right_rotate(index)

        # RL Case
        if balance < -1 and data < self.data[self.right[index]]:
            self.right[index] = self.right_rotate(self.right[index])
            return self.left_rotate(index)

        return index

    # Insert function
    def insert(self, data):
        self.root = self.insert_node(self.root, data)

    # Find minimum node
    def minimum(self, index):
```

```
    current = index

    while self.left[current] != -1:
        current = self.left[current]

    return current

# Delete node
def delete_node(self, index, data):
    if index == -1:
        return -1

    if data < self.data[index]:
        self.left[index] = self.delete_node(
            self.left[index], data
        )

    elif data > self.data[index]:
        self.right[index] = self.delete_node(
            self.right[index], data
        )

    else:
        # No child
        if self.left[index] == -1 and self.right[index] == -1:
            return -1

        # Only right child
        elif self.left[index] == -1:
```

```
            return self.right[index]

        # Only left child
        elif self.right[index] == -1:
            return self.left[index]

        # Two children
        else:
            temp = self.minimum(self.right[index])
            self.data[index] = self.data[temp]

            self.right[index] = self.delete_node(
                self.right[index],
                self.data[temp]
            )

    self.update_height(index)

    balance = self.get_balance(index)

    # LL Case
    if balance > 1 and self.get_balance(self.left[index]) >= 0:
        return self.right_rotate(index)

    # LR Case
    if balance > 1 and self.get_balance(self.left[index]) < 0:
        self.left[index] = self.left_rotate(self.left[index])
        return self.right_rotate(index)
```

```
    # RR Case
    if balance < -1 and self.get_balance(self.right[index]) <= 0:
        return self.left_rotate(index)

    # RL Case
    if balance < -1 and self.get_balance(self.right[index]) > 0:
        self.right[index] = self.right_rotate(self.right[index])
        return self.left_rotate(index)

    return index

# Delete function
def delete(self, data):
    if self.search(data):
        self.root = self.delete_node(self.root, data)
        print("Element deleted")
    else:
        print("Element not found")

# Search node
def search_node(self, index, data):
    if index == -1:
        return False

    if self.data[index] == data:
        return True

    if data < self.data[index]:
        return self.search_node(self.left[index], data)
```

```
    return self.search_node(self.right[index], data)

# Search function
def search(self, data):
    return self.search_node(self.root, data)

# Inorder
def inorder(self, index):
    if index != -1:
        self.inorder(self.left[index])
        print(self.data[index], end=" ")
        self.inorder(self.right[index])

# Preorder
def preorder(self, index):
    if index != -1:
        print(self.data[index], end=" ")
        self.preorder(self.left[index])
        self.preorder(self.right[index])

# Postorder
def postorder(self, index):
    if index != -1:
        self.postorder(self.left[index])
        self.postorder(self.right[index])
        print(self.data[index], end=" ")

# Display array
```

```
    def display(self):
        print("\nIndex\tData\tLeft\tRight\tHeight")

        for i in range(self.count):
            print(
                i, "\t",
                self.data[i], "\t",
                self.left[i], "\t",
                self.right[i], "\t",
                self.height[i]
            )


# Main Program

avl = AVLTree(50)

while True:
    print("\n----- AVL TREE USING ARRAY -----")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter choice: "))
```

```
if choice == 1:
    value = int(input("Enter value: "))
    avl.insert(value)
    print("Element inserted")

elif choice == 2:
    value = int(input("Enter value: "))
    avl.delete(value)

elif choice == 3:
    value = int(input("Enter value: "))

    if avl.search(value):
        print("Element found")
    else:
        print("Element not found")

elif choice == 4:
    print("Inorder:", end=" ")
    avl.inorder(avl.root)
    print()

elif choice == 5:
    print("Preorder:", end=" ")
    avl.preorder(avl.root)
    print()

elif choice == 6:
```

```
        print("Postorder:", end=" ")
        avl.postorder(avl.root)
        print()

    elif choice == 7:
        avl.display()

    elif choice == 8:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

Output:

```
========== RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Pytho

----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 1
Enter value: 5
Element inserted

----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 4
Inorder: 5

----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 6
Postorder: 5
```

```
Enter choice: 1
Enter value: 5
Element inserted

----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 2
Enter value: 6
Element not found

----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 4
Inorder: 2 5 10

----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 5
Preorder: 5 2 10
```

```
----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 6
Postorder: 2 10 5

----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 7

Index   Data    Left    Right   Height
0        2       -1      -1      1
1        10      -1      -1      1
2        5       0       1       2

----- AVL TREE USING ARRAY -----
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter choice: 8
Program ended
>>>
```

**Explanation:**

- An **AVL tree** is a self-balancing Binary Search Tree (BST).
- In an AVL tree, the difference between the height of the left and right subtrees is called the **Balance Factor**.
- The Balance Factor of every node must be **-1, 0, or +1**.
- In the array implementation, the tree nodes are stored inside a fixed-size array.
- For a node at index i:
    - Left child is stored at 2*i + 1.
    - Right child is stored at 2*i + 2.
- None is used to represent an empty position in the array.

**Main Functions**

- **__init__()**
    - Creates the array for storing the tree.
    - The size of the array is given by the user/program.
    - Initially, all positions contain None.
- **height()**
    - Calculates the height of a node.
    - It recursively checks the left and right subtrees.
    - The greater height is selected and 1 is added.
- **balance_factor()**
    - Calculates the balance factor of a node.
    - It is calculated as:
        - Height of Left Subtree - Height of Right Subtree
    - A value greater than 1 indicates that the tree is left-heavy.
    - A value less than -1 indicates that the tree is right-heavy.
- **insert()**
    - Adds a new element to the AVL tree.
    - The element is placed according to Binary Search Tree rules.
    - Smaller values are placed in the left subtree.

    - Larger values are placed in the right subtree.
    - After insertion, the tree is checked and rotations are performed if required.
- **search()**
    - Searches for a particular element.
    - If the element is found, it returns True.
    - If the element does not exist, it returns False.
- **delete()**
    - Removes an element from the tree.
    - It handles nodes with no child, one child, or two children.
    - After deletion, the tree is rebalanced.
- **find_min()**
    - Finds the smallest element in a subtree.
    - It repeatedly moves to the left child.
    - It is mainly used when deleting a node having two children.
- **rotate_left()**
    - Performs a left rotation when the tree becomes right-heavy.
    - It helps restore the AVL property.
- **rotate_right()**
    - Performs a right rotation when the tree becomes left-heavy.
    - It helps restore the AVL property.
- **balance_tree()**
    - Checks the balance factor of the nodes.
    - If the tree becomes unbalanced, suitable rotations are performed.
- **inorder()**
    - Traverses the tree in:
        - Left → Root → Right
    - It displays the elements in sorted order.
- **preorder()**
    - Traverses the tree in:

        - Root → Left → Right
- **postorder()**
    - Traverses the tree in:
        - Left → Right → Root
- **display()**
    - Displays the elements along with their array indexes.
    - It helps to understand how the tree is stored in the array.

**Main Program**

- The main program provides a **menu-driven interface**.
- The user can choose:
    - Insert
    - Delete
    - Search
    - Inorder
    - Preorder
    - Postorder
    - Display
    - Exit
- This allows all major AVL tree operations to be performed interactively.

**Error table:**

| S.No. | Common Error | Reason / Correction |
|---|---|---|
| 1 | Index out of range | The calculated child index exceeds the array size. Use a sufficiently large array and check the index before accessing it. |
| 2 | Tree becomes full | The fixed-size array has no empty position available. Increase the array size. |
| 3 | Incorrect child index | Using the wrong formula for child positions causes incorrect tree structure. Use left = 2*i+1 and right = 2*i+2. |
| 4 | Incorrect balancing after insertion/deletion | The balance factor or rotation is not handled correctly. Check the balance factor and perform the required rotation. |

# Week-10

**Aim:** Write a Python program to implement Hashing using Separate Chaining, Linear Probing, and Quadratic Probing.

**Program:**

```
class Hashing:

    def __init__(self, size):
        self.size = size
        self.chain = [[] for i in range(size)]
        self.linear = [None] * size
        self.quadratic = [None] * size

    # Hash function
    def hash_function(self, key):
        return key % self.size

    # Separate Chaining
    def insert_chaining(self, key):
        index = self.hash_function(key)

        if key not in self.chain[index]:
            self.chain[index].append(key)

    # Linear Probing
    def insert_linear(self, key):
        index = self.hash_function(key)
```

```
    for i in range(self.size):
        pos = (index + i) % self.size

        if self.linear[pos] is None:
            self.linear[pos] = key
            return

        elif self.linear[pos] == key:
            print("Duplicate value")
            return

    print("Hash table is full")

# Quadratic Probing
def insert_quadratic(self, key):
    index = self.hash_function(key)

    for i in range(self.size):
        pos = (index + i * i) % self.size

        if self.quadratic[pos] is None:
            self.quadratic[pos] = key
            return

        elif self.quadratic[pos] == key:
            print("Duplicate value")
            return
```

```
        print("No empty position found")

    # Search using Separate Chaining
    def search_chaining(self, key):
        index = self.hash_function(key)

        return key in self.chain[index]

    # Search using Linear Probing
    def search_linear(self, key):
        index = self.hash_function(key)

        for i in range(self.size):
            pos = (index + i) % self.size

            if self.linear[pos] is None:
                return False

            if self.linear[pos] == key:
                return True

        return False

    # Search using Quadratic Probing
    def search_quadratic(self, key):
        index = self.hash_function(key)

        for i in range(self.size):
            pos = (index + i * i) % self.size
```

```
        if self.quadratic[pos] is None:
            return False

        if self.quadratic[pos] == key:
            return True

    return False

# Display Separate Chaining
def display_chaining(self):
    print("\nSeparate Chaining:")

    for i in range(self.size):
        print(i, ":", self.chain[i])

# Display Linear Probing
def display_linear(self):
    print("\nLinear Probing:")

    for i in range(self.size):
        print(i, ":", self.linear[i])

# Display Quadratic Probing
def display_quadratic(self):
    print("\nQuadratic Probing:")

    for i in range(self.size):
        print(i, ":", self.quadratic[i])
```

```
# Main

h = Hashing(11)

values = [21, 32, 43, 54, 65, 76]

for value in values:
    h.insert_chaining(value)
    h.insert_linear(value)
    h.insert_quadratic(value)

h.display_chaining()
h.display_linear()
h.display_quadratic()

print("\nSearch 43:")

print("Separate Chaining:", h.search_chaining(43))
print("Linear Probing:", h.search_linear(43))
print("Quadratic Probing:", h.search_quadratic(43))
```

**Output:**

```
Enter "help" below or click "Help" above for more information.

= RESTART: C:/Users/Yeswanth/AppData/Local/Programs/Python/Python314/week10.py =

Separate Chaining:
0 : []
1 : []
2 : []
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []
10 : [21, 32, 43, 54, 65, 76]

Linear Probing:
0 : 32
1 : 43
2 : 54
3 : 65
4 : 76
5 : None
6 : None
7 : None
8 : None
9 : None
10 : 21

Quadratic Probing:
0 : 32
1 : None
2 : 76
3 : 43
4 : 65
5 : None
6 : None
```

**Explanation:**

- The Hashing class is used to implement hashing using three collision-resolution techniques: Separate Chaining, Linear Probing, and Quadratic Probing.

- The __init__() method initializes the hash table with a given size. Three different structures are created to store elements using the three techniques.
- The hash_function() calculates the index for a given key using the modulo operator. The formula used is key % size.
- The insert_chaining() function inserts elements into a list at the calculated index. If multiple elements produce the same index, they are stored in the same list.
- The insert_linear() function handles collisions by checking the next available position sequentially until an empty position is found.
- The insert_quadratic() function resolves collisions by checking positions using the square of the iteration number.
- The search_chaining() function searches for an element inside the list at the calculated hash index.
- The search_linear() function searches for an element by checking consecutive positions in the hash table.
- The search_quadratic() function searches for an element using the quadratic probing formula.
- The display_chaining() function displays all the indexes and their corresponding lists.
- The display_linear() function displays all the indexes and values stored using linear probing.
- The display_quadratic() function displays all the indexes and values stored using quadratic probing.
- In the main program, a hash table of size 11 is created, and multiple values are inserted using all three collision-resolution techniques.
- Finally, all three hash tables are displayed, and the search operation is performed to check whether the element 43 exists.

**Error Table:**

| S.No | Common Error | Correction |
|---|---|---|
| 1 | Using an incorrect hash function. | Use key % size to calculate the index. |
| 2 | Not handling collisions during insertion. | Use Separate Chaining, Linear Probing, or Quadratic Probing. |
| 3 | Accessing an index outside the hash table. | Use modulo operation to keep the index within the table size. |
| 4 | Overwriting an existing element during linear probing. | Check whether the position is empty before inserting. |
| 5 | Using an incorrect quadratic probing formula. | Use (index + i*i) % size. |
| 6 | Not checking whether an element exists during searching. | Compare the key with the stored elements and return True or False. |